



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks



Absolute Beginner to Specialization in Apache Spark and Azure Databricks



Spark Data Frame Rows

Spark DataFrame is similar to a table of rows and columns. A row in a DataFrame is a single record represented by an object of type Row. Most of the time, you do not directly work with the entire row. However, we have three specific scenarios when you might have to work with the Row object:

1. Manually creating Rows and DataFrame.
2. Collecting DataFrame rows to the driver.
3. Work with single column Rows in transformations

The first two scenarios are primarily used in unit testing or during development. And we already learned the first scenario in an earlier lecture. Here is a screenshot of the earlier example shown below.

We manually created a list of Row objects and used it to create a DataFrame.

```
Cmd 10

from pyspark.sql import Row

row_list = [Row(a=1, b=2.0, c='string1', d=date(2022, 1, 1), e=datetime(2022, 1, 1, 12, 0)),
            Row(a=2, b=3.0, c='string2', d=date(2022, 2, 1), e=datetime(2022, 1, 2, 12, 0)),
            Row(a=3, b=4.0, c='string3', d=date(2022, 3, 1), e=datetime(2022, 1, 3, 12, 0))]

Cmd 11

spark.createDataFrame(row_list).printSchema()
```

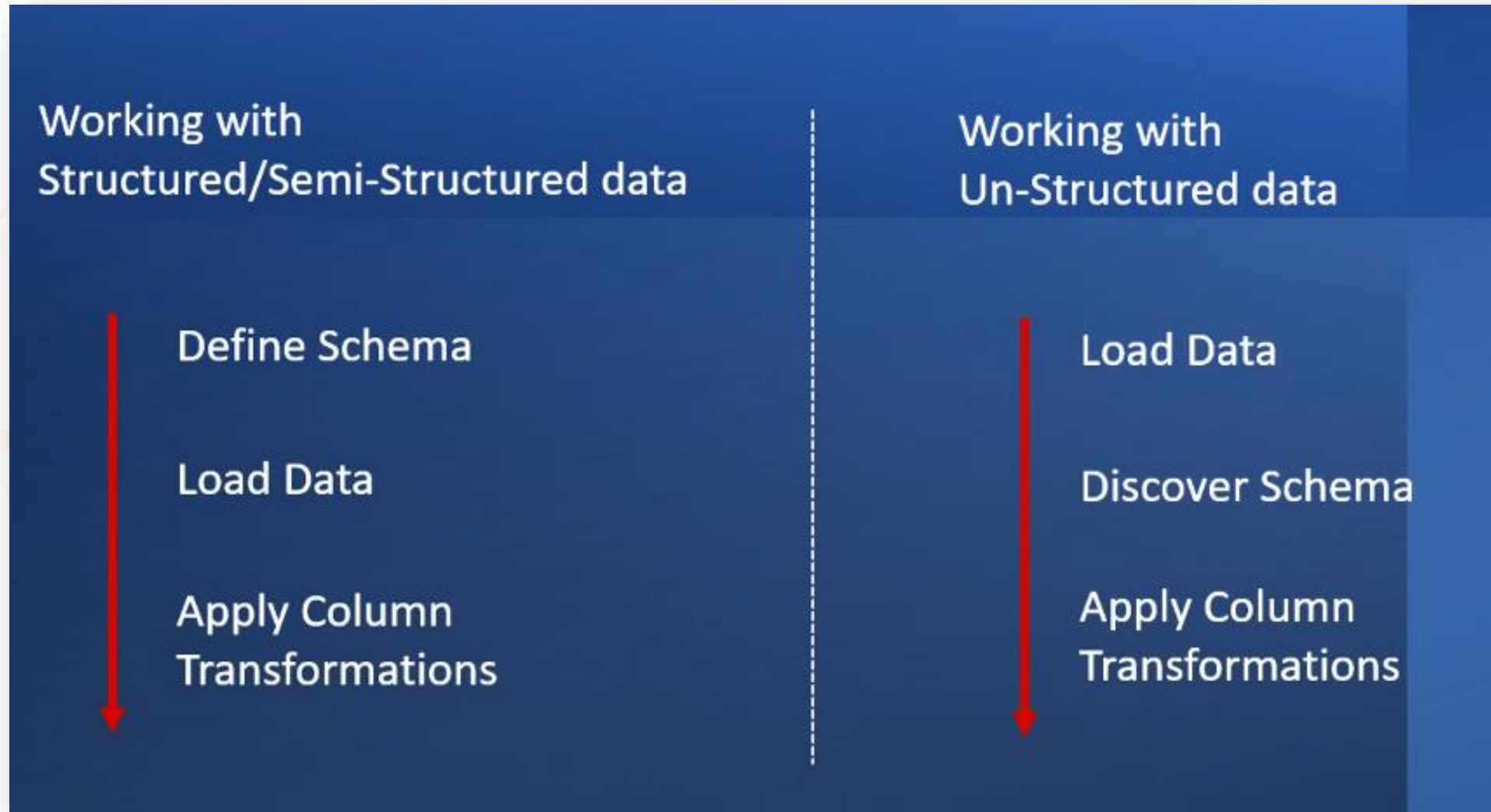
1. Manually creating Rows and Data Frame
2. Collecting Data Frame Rows to the driver
3. Work with single column Row in transformations

The third scenario of “working with single column rows in transformations” is mainly used with unstructured data.

I have this log file shown below. This file contains the application log from the Apache web server. This file contains a lot of information. But we are interested in the HTTP:// type of URLs. These are referrers of our website. They referred users to your website, and you want to take them out from this log and count who sent how many users.

```
83.149.9.216 - - [17/May/2015:10:05:03 +0000] "GET /presentations/logstash-monitorama-2013/images/kibana-search.png HTTP/1.1" 200 203023
"http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML,
Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:43 +0000] "GET /presentations/logstash-monitorama-2013/images/kibana-dashboard3.png HTTP/1.1" 200 171717
"http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML,
Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:47 +0000] "GET /presentations/logstash-monitorama-2013/plugin/highlight/highlight.js HTTP/1.1" 200 26185
"http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML,
Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:12 +0000] "GET /presentations/logstash-monitorama-2013/plugin/zoom-js/zoom.js HTTP/1.1" 200 7697
"http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML,
Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:07 +0000] "GET /presentations/logstash-monitorama-2013/plugin/notes/notes.js HTTP/1.1" 200 2892
"http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML,
Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:34 +0000] "GET /presentations/logstash-monitorama-2013/images/sad-medic.png HTTP/1.1" 200 430406
"http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML,
Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:57 +0000] "GET /presentations/logstash-monitorama-2013/css/fonts/Roboto-Bold.ttf HTTP/1.1" 200 38720
"http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML,
Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:50 +0000] "GET /presentations/logstash-monitorama-2013/css/fonts/Roboto-Regular.ttf HTTP/1.1" 200 41820
"http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML,
Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:24 +0000] "GET /presentations/logstash-monitorama-2013/images/frontend-response-codes.png HTTP/1.1" 200 52
"http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML,
Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:50 +0000] "GET /presentations/logstash-monitorama-2013/images/kibana-dashboard.png HTTP/1.1" 200 321631
"http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML,
Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:46 +0000] "GET /presentations/logstash-monitorama-2013/images/Dreamhost_logo.svg HTTP/1.1" 200 2126
"http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML,
Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:11 +0000] "GET /presentations/logstash-monitorama-2013/images/kibana-dashboard2.png HTTP/1.1" 200 394967
"http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML,
Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:19 +0000] "GET /presentations/logstash-monitorama-2013/images/apache-icon.gif HTTP/1.1" 200 8095
"http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML,
Chrome/32.0.1700.77 Safari/537.36"
```

Spark allows you to work with structured/semi-structured and unstructured data. However, the approach is different as shown below.



When you have structured or semi-structured data, you will define the schema of your data source.

Load it to a Spark Dataframe.

And start applying transformations.

But what if you have unstructured data.

The approach is different.

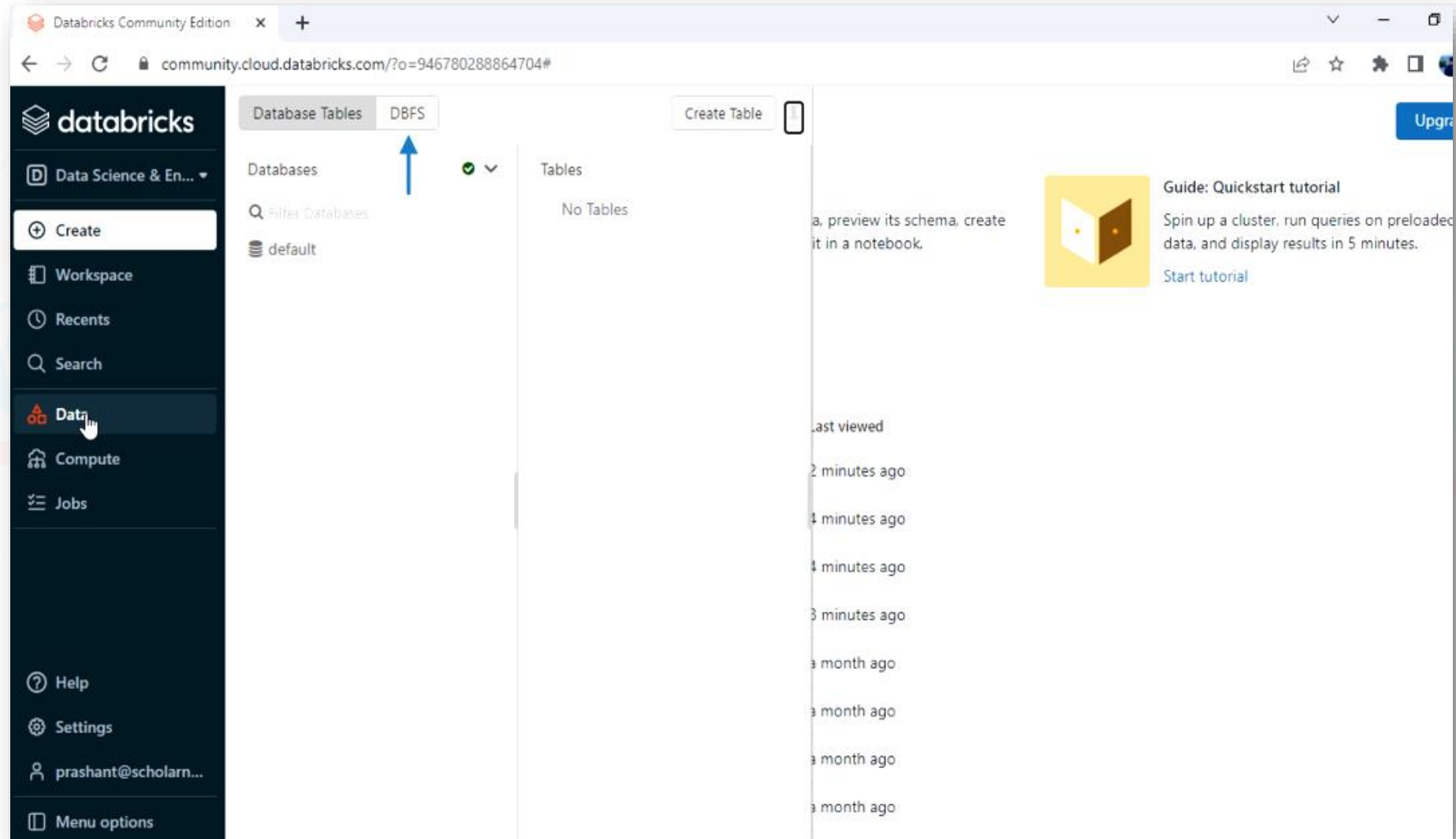
Load the unstructured data into a Dataframe. You won't have a meaningful schema.

It is most likely a line of text in each row.

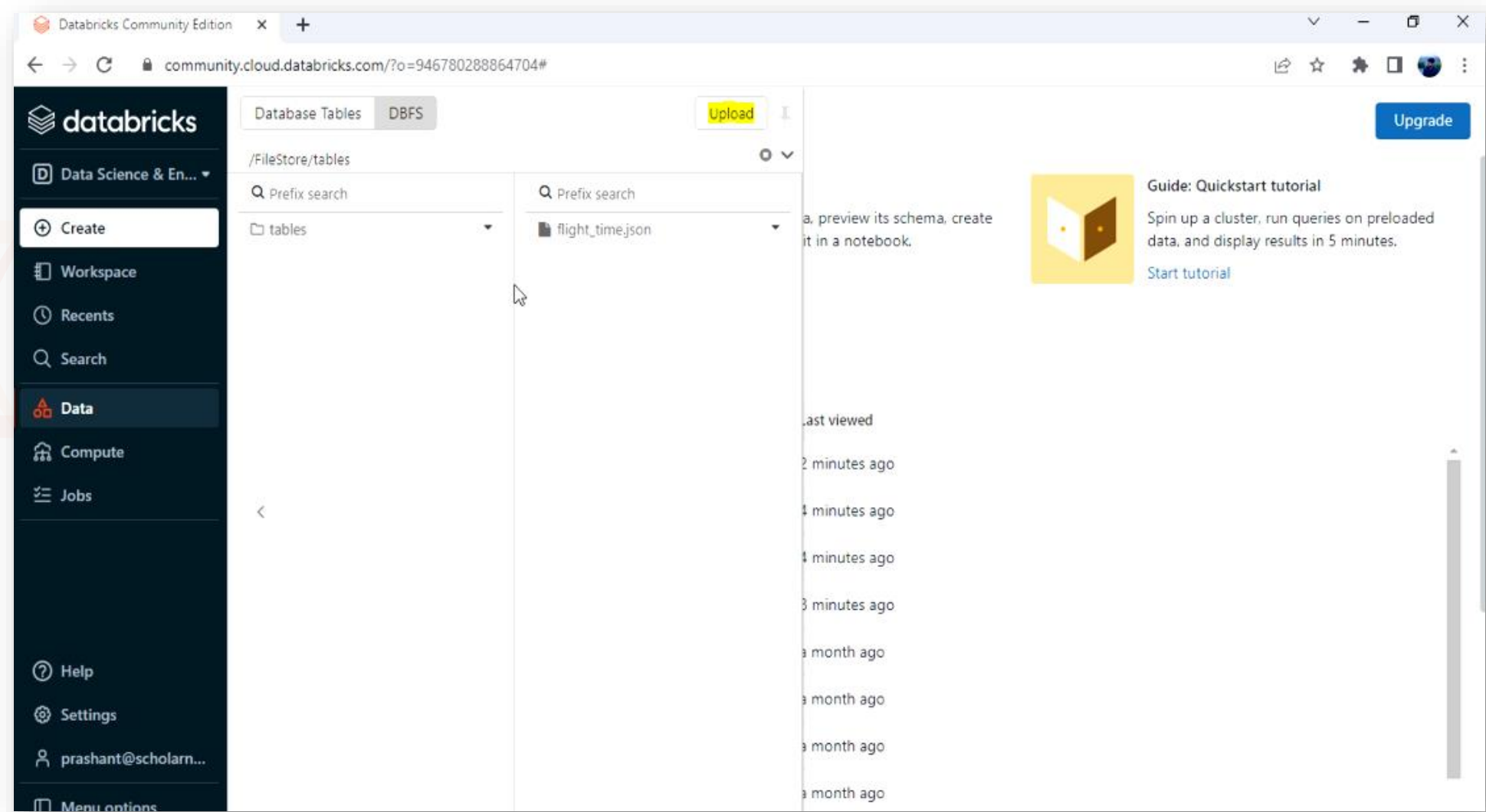
But then you can discover a schema and give your Dataframe a row/column structure.

Basically, we convert the unstructured data into structured and start transforming and analysing it.

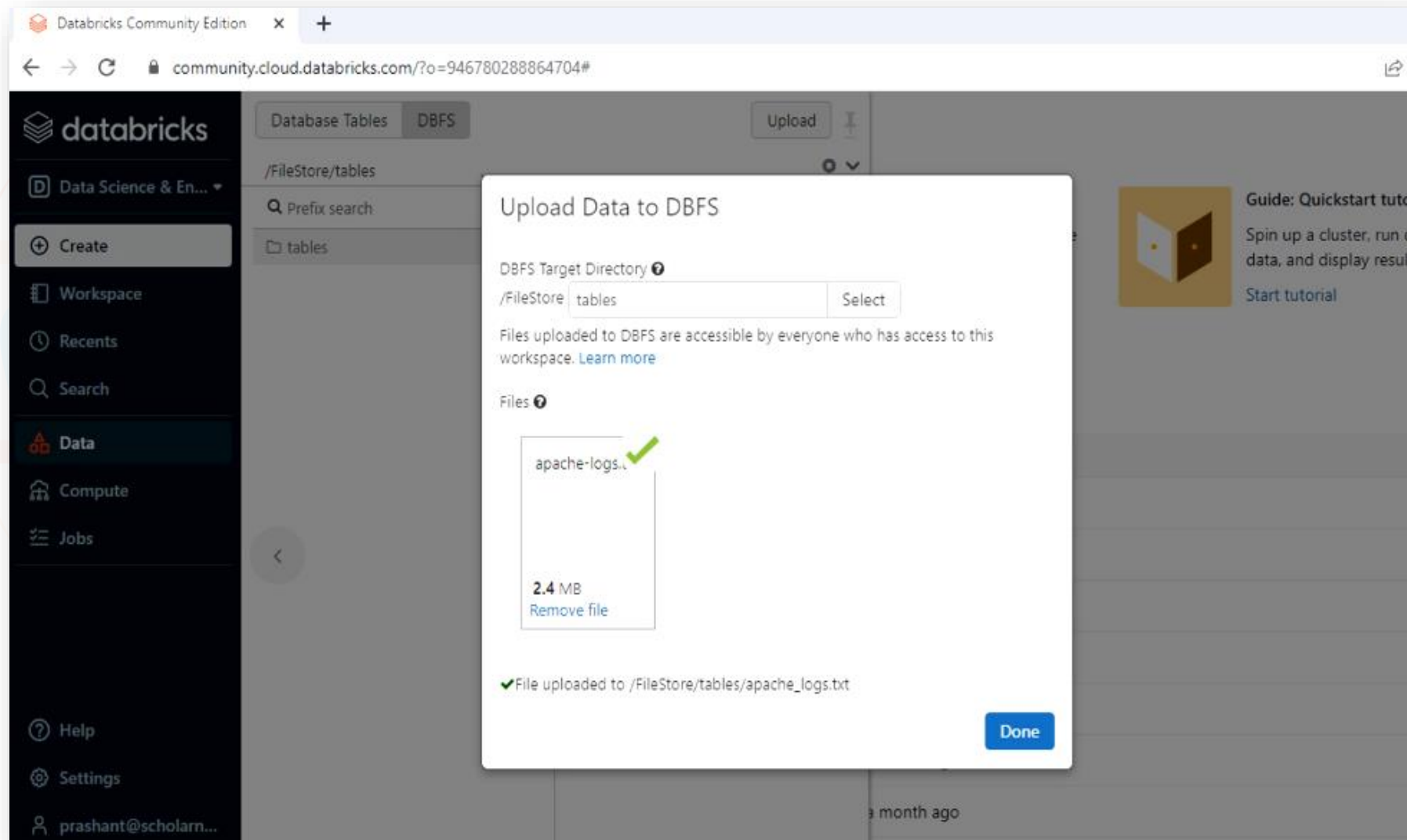
Go to your data bricks workspace and upload the log file into DBFS.
Go to the Data menu. And you will see the DBFS tab. Click that, and you can navigate to the filesystem.



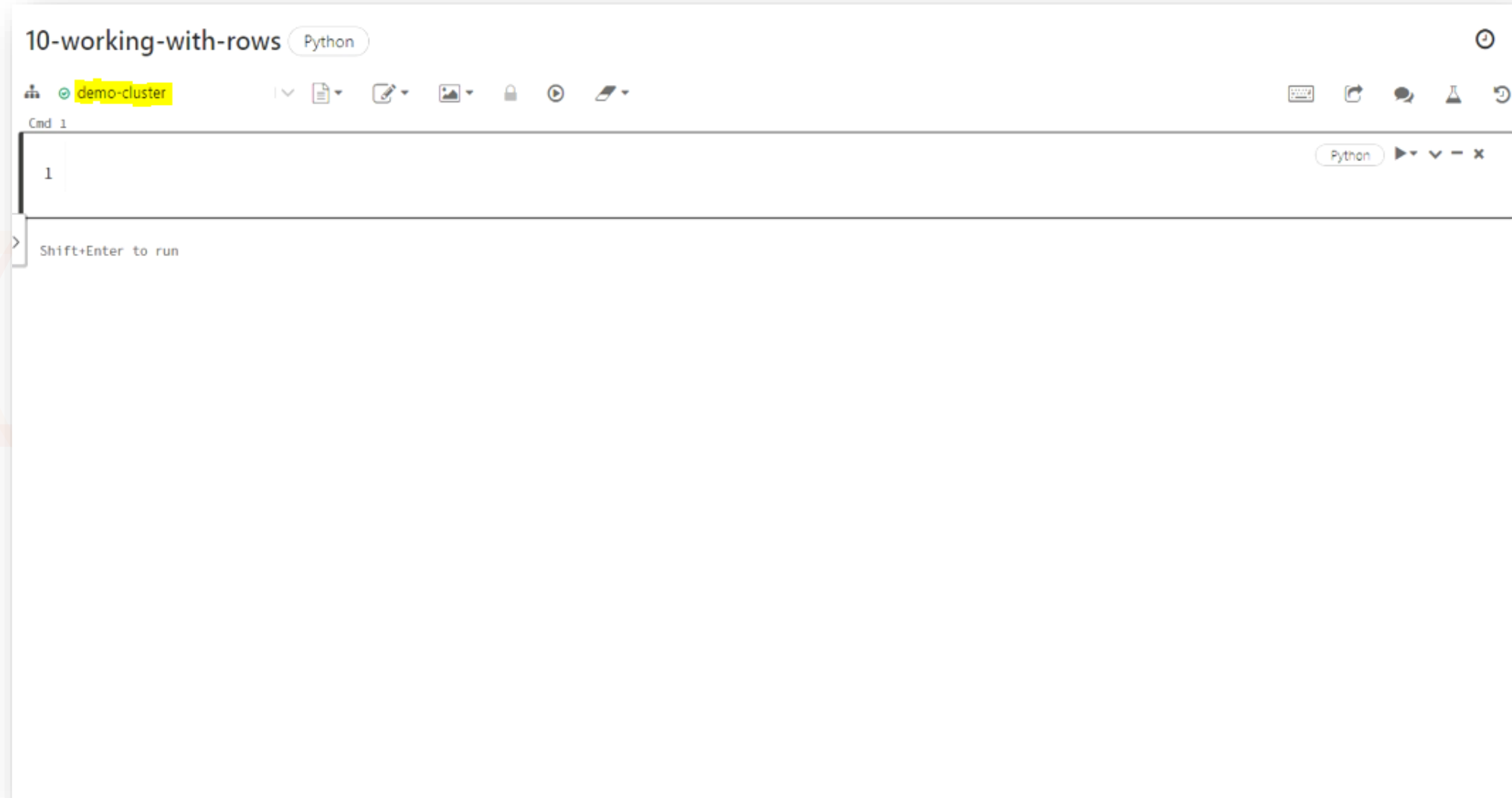
Select Filestore/tables and then click the upload button. And it will show the DBFS target directory here. You can click the upload option and choose your data file to upload.



My data file is in the cloud, and we are now ready to start processing it.



Create a new notebook. **(Reference : 10-working-with-rows)**
Make sure you attach a running cluster to your notebook.



We have already uploaded the log file to the cloud storage. You can use the command shown below to see the file.

The next step is to load this file into a Spark DataFrame.

10-working-with-rows

Python

demo-cluster

Cmd 1

1 %fs ls /FileStore/tables/

	path	name	size	modificationTime
1	dbfs:/FileStore/tables/apache_logs.txt	apache_logs.txt	2380042	1651731227000
2	dbfs:/FileStore/tables/flight_time.json	flight_time.json	87150442	1650995258000

Showing all 2 rows.

Command took 7.08 seconds -- by prashant@scholarnest.com at 5/5/2022, 11:45:50 AM on demo-cluster

Cmd 2

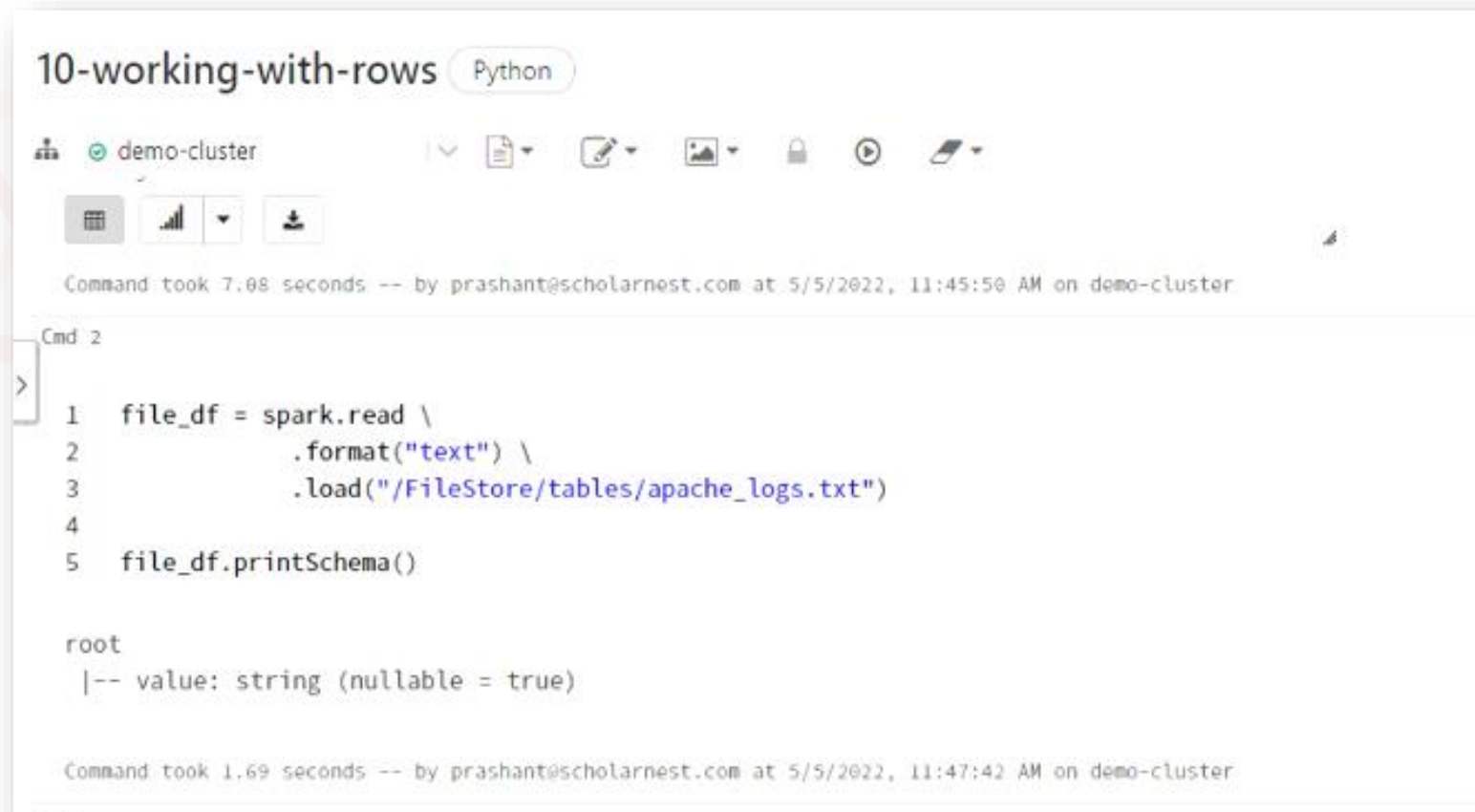
1

Shift+Enter to run

Here is the code to load the file into Spark Dataframe.

We start with the spark session variable, use the read attribute to get the DataFrameReader, set the format, and load it. The format is text. Then, I am also printing the schema of the data frame.

And you can see the output for this code below the cell. Dataframe must have a schema. You cannot create a schema-less Dataframe. We are loading a text file which is an unstructured data file. However, the Dataframe must have a schema even if you load an unstructured data file.



The screenshot shows a Databricks notebook interface. At the top, the tab is labeled "10-working-with-rows" with a "Python" language selector. Below the tab, the cluster name "demo-cluster" is visible. The command execution history shows a previous command took 7.08 seconds. The current command, labeled "Cmd 2", contains the following Python code:

```
1 file_df = spark.read \  
2     .format("text") \  
3     .load("/FileStore/tables/apache_logs.txt") \  
4 \  
5 file_df.printSchema()
```

The output of the command shows the schema:

```
root \  
|-- value: string (nullable = true)
```

The command execution history at the bottom shows this command took 1.69 seconds.

Here is the Dataframe. We have a single-column data frame. The column name is value. Each record comes with one line of text, and it is an unstructured Dataframe. Now we want to do some analysis on this data and apply Dataframe transformations. However, most of the Dataframe transformations are applied to columns. And we do not have any column except the value column, because we have unstructured data.

10-working-with-rows Python

demo-cluster

```
1 display(file_df)
```

(1) Spark Jobs

	value
1	83.149.9.216 - - [17/May/2015:10:05:03 +0000] "GET /presentations/logstash-monitorama-2013/images/kibana-search.png HTTP/1.1" 200 203023 "http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"
2	83.149.9.216 - - [17/May/2015:10:05:43 +0000] "GET /presentations/logstash-monitorama-2013/images/kibana-dashboard3.png HTTP/1.1" 200 171717 "http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"
3	83.149.9.216 - - [17/May/2015:10:05:47 +0000] "GET /presentations/logstash-monitorama-2013/plugin/highlight/highlight.js HTTP/1.1" 200 26185 "http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"
4	83.149.9.216 - - [17/May/2015:10:05:12 +0000] "GET /presentations/logstash-monitorama-2013/plugin/zoom-js/zoom.js HTTP/1.1" 200 7697 "http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"
5	83.149.9.216 - - [17/May/2015:10:05:07 +0000] "GET /presentations/logstash-monitorama-2013/plugin/notes/notes.js HTTP/1.1" 200 2892 "http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"
6	83.149.9.216 - - [17/May/2015:10:05:34 +0000] "GET /presentations/logstash-monitorama-2013/images/sad-medic.png HTTP/1.1" 200 430406 "http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"
	83.149.9.216 - - [17/May/2015:10:05:57 +0000] "GET /presentations/logstash-monitorama-2013/css/fonts/Roboto-Bold.ttf

Truncated results, showing first 1000 rows.

I gave some structure to the data, by taking one record, and breaking it with spaces.

So I identified 11 fields in this data. The first one is the IP address. The second and third ones are hyphens. The fourth one is a date. Then we have Get or Post method. The next one is an absolute file name. Then we have protocol followed by two numbers. After all this, we have the referrer URL I am looking for. Once I get a URL, I do not care what else is there, so consider it one field.

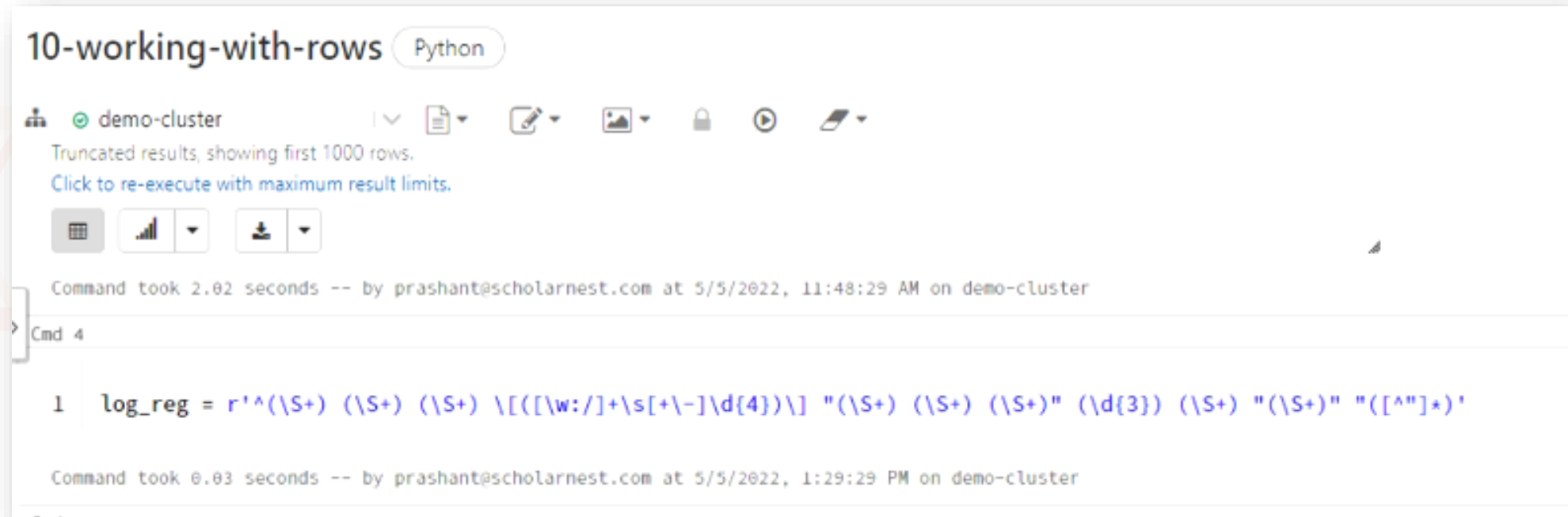
```
83.149.9.216 - - [17/May/2015:10:05:03 +0000] "GET /presentations/logstash-monitorama-2013/images/kibana-search.png HTTP/1.1" 200 203023  
"http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0  
(Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko)  
Chrome/32.0.1700.77 Safari/537.36"
```

1. 83.149.9.216
2. -
3. -
4. [17/May/2015:10:05:03 +0000]
5. "GET
6. /presentations/logstash-monitorama-2013/images/kibana-search.png
7. HTTP/1.1"
8. 200
9. 203023
10. "http://semicomplete.com/presentations/logstash-monitorama-2013/"
11. "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"

I saw the data and realized that the referrer's URL is in the file. However, we have another nine logical fields before the referrer. So If I can break this text line into eleven fields as listed here, I can put a structure to the given unstructured Dataframe. It is not easy to split this text using space characters. I thought of applying regular expressions. And here are eleven expressions assigned for each field.

<pre>83.149.9.216 - - [17/May/2015:10:05:03 +0000] "GET /presentations/logstash-monitorama-2013/images/kibana-search.png HTTP/1.1" 200 203023 "http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"</pre>	
<pre>1. 83.149.9.216 2. - 3. - 4. [17/May/2015:10:05:03 +0000] 5. "GET 6. /presentations/logstash-monitorama-2013/images/kibana-search.png 7. HTTP/1.1" 8. 200 9. 203023 10. "http://semicomplete.com/presentations/logstash-monitorama-2013/" 11. "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"</pre>	<pre>1. (\S+) 2. (\S+) 3. (\S+) 4. \[([\\w:/]+\\s[+\\-]\\d{4})\\] 5. "(\\S+) 6. (\\S+) 7. (\\S+)" 8. (\\d{3}) 9. (\\S+) 10. "(\\S+)" 11. "([^"]*)"</pre>

Here is the regular expression that represents all the eleven fields.



The screenshot shows a Jupyter Notebook interface with a title bar "10-working-with-rows" and a "Python" language selector. Below the title bar, there is a toolbar with icons for saving, running, and other actions. The main area displays a code cell with the following content:

```
log_reg = r'^(\S+) (\S+) (\S+) \[([w:/]+\s[+-]\d{4})\] "(\S+) (\S+) (\S+)" (\d{3}) (\S+) "(\S+)" "([^\"]*)'
```

Below the code cell, there is a status bar indicating the command took 0.03 seconds to execute.

Spark gives us `regexp_extract()` function to extract one field from a regular expression, and I have the code for that shown below. So I am using the `select()` method to take out four fields. The `regexp_extract()` method takes three arguments. The first one is the column on which you want to apply the regular expression. The second argument is the regular expression itself. The last argument is the position after splitting the value using the regular expression.

10-working-with-rows

Python

demo-cluster

Cmd 5

```
1 from pyspark.sql.functions import regexp_extract
2
3 logs_df = file_df.select(regexp_extract('value', log_reg, 1).alias('ip'),
4                           regexp_extract('value', log_reg, 4).alias('date'),
5                           regexp_extract('value', log_reg, 6).alias('image'),
6                           regexp_extract('value', log_reg, 10).alias('referrer'))
7
8 display(logs_df)
```

(1) Spark Jobs

	ip	date	image	referrer
1	83.149.9.216	17/May/2015:10:05:03 +0000	/presentations/logstash-monitorama-2013/images/kibana-search.png	http://semicom
2	83.149.9.216	17/May/2015:10:05:43 +0000	/presentations/logstash-monitorama-2013/images/kibana-dashboard3.png	http://semicom
3	83.149.9.216	17/May/2015:10:05:47 +0000	/presentations/logstash-monitorama-2013/plugin/highlight/highlight.js	http://semicom
4	83.149.9.216	17/May/2015:10:05:12 +0000	/presentations/logstash-monitorama-2013/plugin/zoom-js/zoom.js	http://semicom
5	83.149.9.216	17/May/2015:10:05:07 +0000	/presentations/logstash-monitorama-2013/plugin/notes/notes.js	http://semicom
6	83.149.9.216	17/May/2015:10:05:34 +0000	/presentations/logstash-monitorama-2013/images/sad-medic.png	http://semicom
7	83.149.9.216	17/May/2015:10:05:57 +0000	/presentations/logstash-monitorama-2013/css/fonts/Roboto-Bold.ttf	http://semicom

Truncated results, showing first 1000 rows.
[Click to re-execute with maximum result limits.](#)

Command took 0.84 seconds -- by prashant@scholarnest.com at 5/5/2022, 1:50:56 PM on demo-cluster

I know the first position is for IP address, the fourth is a date, the sixth is the image, and the tenth field is the referrer. I am only interested in a referrer. But I tried to extract four fields so you can learn how to extract multiple files.

10-working-with-rows

Python

demo-cluster

Cmd 5

```
1 from pyspark.sql.functions import regexp_extract
2
3 logs_df = file_df.select(regexp_extract('value', log_reg, 1).alias('ip'),
4                           regexp_extract('value', log_reg, 4).alias('date'),
5                           regexp_extract('value', log_reg, 6).alias('image'),
6                           regexp_extract('value', log_reg, 10).alias('referrer'))
7
8 display(logs_df)
```

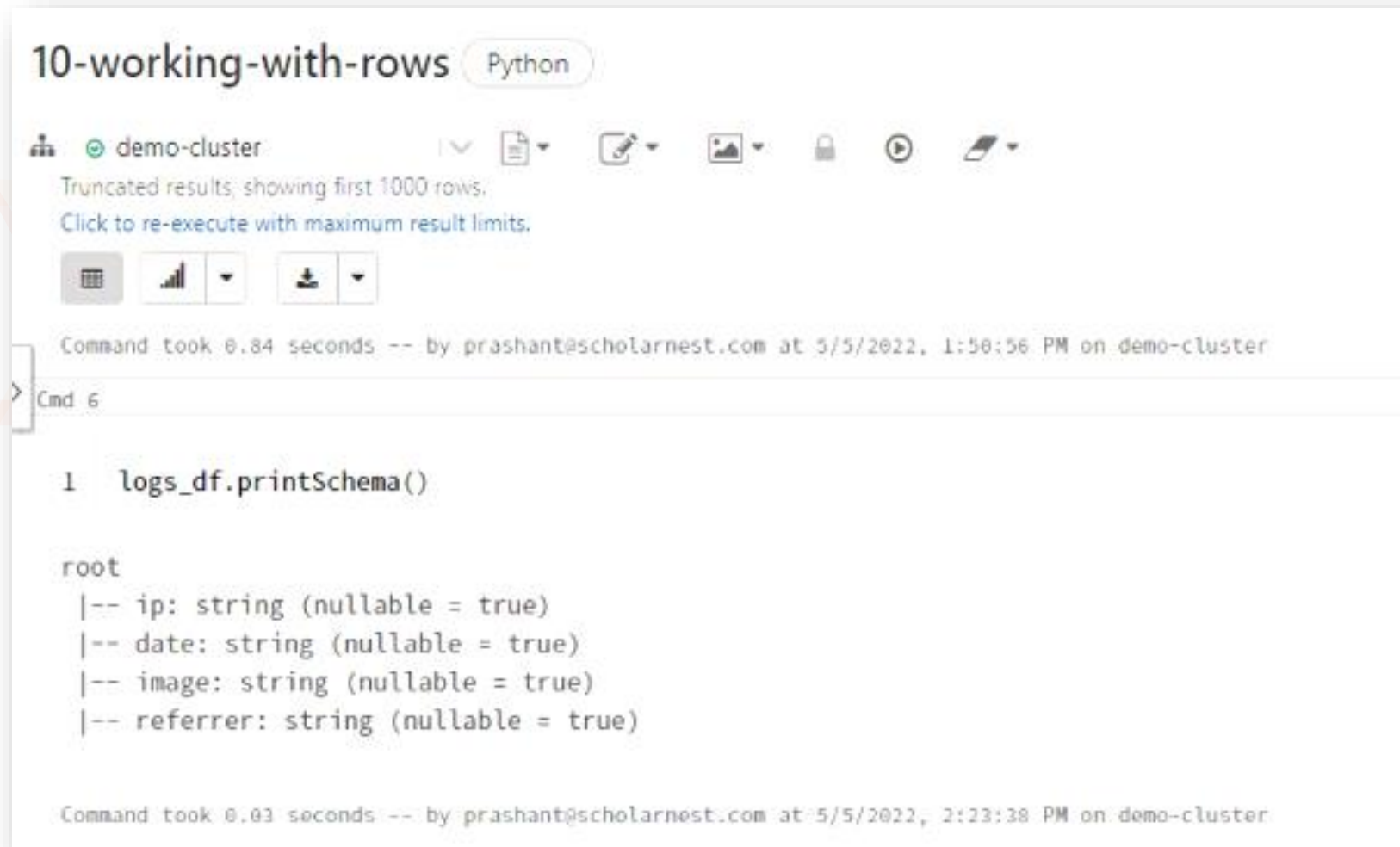
(1) Spark Jobs

	ip	date	image	referrer
1	83.149.9.216	17/May/2015:10:05:03 +0000	/presentations/logstash-monitorama-2013/images/kibana-search.png	http://semicom
2	83.149.9.216	17/May/2015:10:05:43 +0000	/presentations/logstash-monitorama-2013/images/kibana-dashboard3.png	http://semicom
3	83.149.9.216	17/May/2015:10:05:47 +0000	/presentations/logstash-monitorama-2013/plugin/highlight/highlight.js	http://semicom
4	83.149.9.216	17/May/2015:10:05:12 +0000	/presentations/logstash-monitorama-2013/plugin/zoom-js/zoom.js	http://semicom
5	83.149.9.216	17/May/2015:10:05:07 +0000	/presentations/logstash-monitorama-2013/plugin/notes/notes.js	http://semicom
6	83.149.9.216	17/May/2015:10:05:34 +0000	/presentations/logstash-monitorama-2013/images/sad-medic.png	http://semicom
7	83.149.9.216	17/May/2015:10:05:57 +0000	/presentations/logstash-monitorama-2013/css/fonts/Roboto-Bold.ttf	http://semicom

Truncated results, showing first 1000 rows.
[Click to re-execute with maximum result limits.](#)

Command took 0.84 seconds -- by prashant@scholarnest.com at 5/5/2022, 1:50:56 PM on demo-cluster

Here is the schema definition for the Dataframe. Now I can apply some column transformations.



The screenshot shows a Databricks workspace interface. At the top, the title is "10-working-with-rows" with a "Python" language selector. Below the title, there's a toolbar with icons for cluster, file, edit, view, lock, refresh, and help. The cluster name is "demo-cluster". A message states "Truncated results: showing first 1000 rows. Click to re-execute with maximum result limits." Below this are icons for table, bar chart, and download. The command history shows a command executed on 5/5/2022 at 1:50:56 PM, which took 0.84 seconds. The command is "Cmd 6" and the output is the schema of the "logs_df" dataframe. The schema shows four columns: "ip", "date", "image", and "referrer", all of type "string" and nullable. A second command history entry shows a command executed on 5/5/2022 at 2:23:38 PM, which took 0.03 seconds.

```
10-working-with-rows Python
```

demo-cluster

Truncated results: showing first 1000 rows.
Click to re-execute with maximum result limits.

Command took 0.84 seconds -- by prashant@scholarnest.com at 5/5/2022, 1:50:56 PM on demo-cluster

Cmd 6

```
1 logs_df.printSchema()

root
 |-- ip: string (nullable = true)
 |-- date: string (nullable = true)
 |-- image: string (nullable = true)
 |-- referrer: string (nullable = true)
```

Command took 0.03 seconds -- by prashant@scholarnest.com at 5/5/2022, 2:23:38 PM on demo-cluster

Now I have a structured Dataframe. I can easily group by the referrer, take a count, and display it.

You can see the results. And we now know who referred how many clicks.

10-working-with-rows

Python

demo-cluster

Cmd 7

```
1 referrer_df = logs_df.groupBy("referrer") \
2     .count()
3
4 display(referrer_df)
```

(2) Spark Jobs

	referrer	count
1	http://www.semicomplete.com/blog/tags/wifi	1
2	http://manpages.ubuntu.com/manpages/lucid/man1/xdotool.1.html	2
3	http://s-chassis.co.nz/viewtopic.php?f=16&t=9265&sid=fad3ad046ddaa63e65cda6e4e0033c19&start=200	1
4	https://www.google.sk/	1
5	http://www.semicomplete.com/projects/keynav/	65
6	http://superuser.com/questions/355151/is-it-possible-to-remap-the-back-forward-keys-of-a-thinkpad-usb-keyboard-on-linu	1
7	https://www.google.ca/	3

Showing all 627 rows.

Command took 1.94 seconds -- by prashant@scholarnest.com at 5/5/2022, 2:33:45 PM on demo-cluster

We have two problems in this Dataframe as listed below.

1. Look at the results, and you will see the same website not being aggregated together. I mean, these two URLs are the same websites. However, they are aggregating as two different websites.

10-working-with-rows Python

demo-cluster

Cmd 7

```
1 referrer_df = logs_df.groupBy("referrer") \
2   .count()
3
4 display(referrer_df)
```

→ 1. The same website not being aggregated together.
2. You will see nulls or hyphens in the referrer field.

▶ (2) Spark Jobs

	referrer	count
1	http://www.semicomplete.com/blog/tags/wifi	1
2	http://manpages.ubuntu.com/manpages/lucid/man1/xdotool.1.html	2
3	http://s-chassis.co.nz/viewtopic.php?f=16&t=9265&sid=fad3ad046ddaa63e65cda6e4e0033c19&start=200	1
4	https://www.google.sk/	1
5	http://www.semicomplete.com/projects/keynav/	65
6	http://superuser.com/questions/355151/is-it-possible-to-remap-the-back-forward-keys-of-a-thinkpad-usb-keyboard-on-linu	1
7	https://www.google.ca/	3

Showing all 627 rows.

Command took 1.94 seconds -- by prashant@scholarnest.com at 5/5/2022, 2:33:45 PM on demo-cluster

2. I have another problem with this data. Scroll down, and you will see nulls or hyphens in the referrer field. Here are 4K plus hits that are not coming from any referrer.

10-working-with-rowsPython

demo-cluster

Cmd 7

Python

```
1 referrer_df = logs_df.groupBy("referrer") \
2   .count()
3
4 display(referrer_df)
```

(2) Spark Jobs

	referrer	count
309	http://www.google.co.in/url?sa=t&rct=j&q=&esrc=s&source=web&cd=4&ved=0CDQQFjAD&url=http%3A%2F%2Fsemicomplete.com%2Ffiles%2Flogstash%2F&ei=BF4CU4OEIYaiIAf184CABw&usg=AFQjCNE6iuqm7xsu8XzgkypI8Z17jGTzQ&bvm=bv.61535280,d.aGc&cad=rja	1
310	-	4073
311	http://www.google.com/url?sa=t&rct=j&q=&esrc=s&source=web&cd=1&ved=0CCUQFjAA&url=http%3A%2F%2Fwww.semicomplete.com%2Fprojects%2Fxdotool%2F&ei=cZABU8nwApXroASpkICADA&usg=AFQjCNE3V_aCf3-gfNcbS924S6jZ6FqffA&sig2=Gkp9GH13hT3n4yftzrphjg&bvm=bv.61535280,d.cGU	1

Showing all 627 rows.

Command took 1.94 seconds -- by prashant@scholarnest.com at 5/5/2022, 2:33:45 PM on demo-cluster

Cmd 8

Can you solve these problems listed below and give us the top five referrers? I leave it as an exercise for you.

The solution is included in the notebook source code. So you can refer to it.

1. The same website not being aggregated together.
2. You will see nulls or hyphens in the referrer field.
3. Display the top five referrers.



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com